API Templates

Overview & Authentication

This API powers a full-featured e-commerce platform offering product browsing, cart management, order processing, user accounts, and product reviews. All endpoints return **JSON** and accept **JSON** request bodies.

Base URL

```
http://localhost:3000/api
```

Authentication

Protected routes require a **JWT Bearer Token** in the Authorization header. Obtain a token via `POST /api/auth/login`. Tokens expire after **24 hours**.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Roles

Role	Description
`user`	Default role. Can browse products, manage their own cart, place orders, and write reviews.
`admin`	Full access. Can manage products, categories, all orders, all users, and delete reviews.

Auth Endpoints — /api/auth

POST /api/auth/register

Create a new user account. No authentication required.

Request Body

Parameter	Type	Required	Description
`name`	string	Yes	Full name of the user.
`email`	string	Yes	Valid email address. Case-insensitive; stored in lowercase.
`password`	string	Yes	Plain-text password. Hashed with bcrypt + pepper before storage.

Sample Request

```
POST /api/auth/register
Content-Type: application/json

{
  "name": "Ahmed Hassan",
  "email": "ahmed@example.com",
  "password": "SecurePass123"
}
```

Sample Response — 201 Created

```
{
  "msg": "User registered successfully",
  "user": {
    "id": "6650f1a2b3c4d5e6f7a8b9c0",
    "name": "Ahmed Hassan",
    "email": "ahmed@example.com"
  }
}
```

POST /api/auth/login

Request Body

Parameter	Type	Required	Description
`email`	string	Yes	Registered email. Case-insensitive.
`password`	string	Yes	Account password.

Sample Request

```
POST /api/auth/login

{
  "email": "ahmed@example.com",
  "password": "SecurePass123"
}
```

Sample Response — 200 OK

```
{
  "msg": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

> ⚠️ **Note:** Store the token securely (e.g., HTTP-only cookie or secure storage). Include it as a Bearer token in the Authorization header for all protected requests.

User Endpoints — /api/users

Method	Path	Description	Auth	Role
GET	`/api/users/profile`	Get the current user's profile	Required	user
PUT	`/api/users/profile`	Update name, email, or password	Required	user
GET	`/api/users/`	List all users	Required	admin
DELETE	`/api/users/:id`	Delete a user by ID	Required	admin

PUT /api/users/profile — Request Body

Parameter	Type	Required	Description
`name`	string	No	New display name.
`email`	string	No	New email address.
`oldPassword`	string	Conditional	Required when changing password. Must match the current password.
`newPassword`	string	No	New password. Requires `oldPassword` to also be provided.

Sample Response — 200 OK (GET /profile)

```
{
  "_id":  "6650f1a2b3c4d5e6f7a8b9c0",
  "name": "Ahmed Hassan",
  "email": "ahmed@example.com",
  "role":  "user",
  "createdAt": "2025-05-01T10:00:00.000Z"
}
```

Product Endpoints — /api/products

Method	Path	Description	Auth	Role
GET	`/api/products/`	List / search / filter products	None	—

Method	Path	Description	Auth	Role
GET	`/api/products/:id`	Get a single product by ID	None	—
POST	`/api/products/`	Create a new product	Required	admin
PUT	`/api/products/:id`	Update an existing product	Required	admin
DELETE	`/api/products/:id`	Delete a product	Required	admin

Query Parameters — GET /api/products/

Parameter	Type	Required	Description
`search`	string	No	Case-insensitive regex search on product name.
`category`	string	No	Filter by category ObjectId.

Create / Update — Request Body

Parameter	Type	Required	Description
`name`	string	Yes*	Product name. (* required on create)
`price`	number	Yes*	Product price. (* required on create)
`description`	string	No	Product description.
`image`	string	No	URL to product image.
`category`	string	No	Category ObjectId.
`stock`	number	No	Available inventory count.
`material`	string	No	Product material / composition.

Sample Response — GET /api/products/:id

```
{
  "_id":         "665abc123def456ghi789",
  "name":        "Classic Wooden Frame",
  "description": "Handcrafted oak photo frame.",
  "price":       29.99,
  "image":       "https://cdn.yourstore.com/images/frame-oak.jpg",
  "stock":       42,
  "material":    "Oak Wood",
  "category":    { "_id": "665cat001", "name": "Art & Culture" }
}
```

Category Endpoints — /api/categories

Categories are seeded automatically on server start if the collection is empty.

Default categories: Cars, Music, Sports, Series, Art & Culture, Quotes

GET /api/categories/

No authentication required. No query parameters.

Sample Response — 200 OK

```
[
  { "_id": "665cat001", "name": "Cars",          "description": "High-sp
eed thrills..." },
  { "_id": "665cat002", "name": "Music",         "description": "Celebra
te the rhythm..." },
  { "_id": "665cat003", "name": "Sports",        "description": "Capture
the energy..." },
  { "_id": "665cat004", "name": "Series",        "description": "From bi
nge-worthy..." },
  { "_id": "665cat005", "name": "Art & Culture", "description": "A curat
ed collection..." },
  { "_id": "665cat006", "name": "Quotes",        "description": "Words t
hat move..." }
]
```

Cart Endpoints — /api/cart

All cart endpoints require authentication. Each user has at most one cart document. The cart is automatically deleted after a successful checkout.

Method	Path	Description	Auth	Role
GET	`/api/cart/`	Get the current user's cart	Required	user
POST	`/api/cart/`	Add a product to the cart	Required	user
PUT	`/api/cart/:productId`	Update quantity of a cart item	Required	user
DELETE	`/api/cart/:productId`	Remove a specific item from cart	Required	user
DELETE	`/api/cart/`	Clear (delete) the entire cart	Required	user

POST /api/cart/ — Request Body

Parameter	Type	Required	Description
`productId`	string	Yes	ObjectId of the product to add.
`quantity`	number	No	Number of units to add. Defaults to 1. Must be ≥ 1.

Sample Response — GET /api/cart/

```
{
  "_id":  "cartDocId",
  "user": "userId",
  "items": [
    {
      "product": {
        "_id":      "productId",
        "name":     "Classic Wooden Frame",
        "price":    29.99,
        "image":    "https://...",
        "stock":    42,
        "material": "Oak Wood"
      },
      "quantity": 2
    }
  ],
  "totalPrice": 59.98
}
```

Order Endpoints — /api/orders

When an order is marked **delivered**, product stock is decremented automatically. The user's cart is cleared immediately after a successful checkout.

Method	Path	Description	Auth	Role
POST	`/api/orders/checkout`	Place an order from the cart	Required	user
GET	`/api/orders/my-orders`	Get the current user's orders	Required	user
GET	`/api/orders/`	Get all orders	Required	admin
PUT	`/api/orders/:id/status`	Update order status	Required	admin

POST /api/orders/checkout — Request Body

Parameter	Type	Required	Description
`shippingAddress`	string/object	Yes	Delivery address for the order.

PUT /api/orders/:id/status — Request Body

Parameter	Type	Required	Description
`status`	string	Yes	Accepted values: `pending`, `processing`, `shipped`, `delivered`, `cancelled`.

> ⚠️ **Side effect:** Setting status to `delivered` decrements the stock of each ordered product by the ordered quantity.

Sample Response — POST /api/orders/checkout (201)

```
{
  "_id":             "orderDocId",
  "user":            "userId",
  "items": [
    { "product": "productId", "quantity": 2, "price": 29.99 }
  ],
  "totalPrice":      59.98,
  "status":          "pending",
  "shippingAddress": "123 Nile St, Cairo, Egypt",
  "createdAt":       "2025-05-15T09:30:00.000Z"
}
```

Review Endpoints — /api/reviews

Method	Path	Description	Auth	Role
POST	`/api/reviews/`	Submit a product review	Required	user
GET	`/api/reviews/product/:productId`	Get all reviews for a product	None	—
DELETE	`/api/reviews/:id`	Delete a review	Required	admin

POST /api/reviews/ — Request Body

Parameter	Type	Required	Description
`productId`	string	Yes	ObjectId of the product being reviewed.
`rating`	number	Yes	Numeric rating (e.g., 1–5).

Parameter	Type	Required	Description
`comment`	string	No	Optional review text.

Sample Response — GET /api/reviews/product/:productId

```
[
  {
    "_id":     "reviewId",
    "user":    { "_id": "userId", "name": "Ahmed Hassan", "profilePictur
e": "..." },
    "product": "productId",
    "rating":  5,
    "comment": "Amazing quality! Exactly as described.",
    "createdAt": "2025-05-10T14:20:00.000Z"
  }
]
```

Error Handling & Status Codes

All errors return a JSON body with at minimum a `message` field. In development mode, a `stack` field is also included.

Status Code	Meaning	When Returned
200 OK	Success	Standard successful GET / PUT response.
201 Created	Resource created	POST to create a user, product, order, or review.
400 Bad Request	Validation error	Missing required fields, duplicate record, incorrect password, or empty cart.
401 Unauthorized	Auth failure	Missing, expired, or invalid JWT token.
403 Forbidden	Insufficient role	Authenticated user lacks admin privileges.
404 Not Found	Resource not found	Product, order, user, or cart item does not exist.
500 Internal Server Error	Server error	Unexpected server-side exception.

Special Error Cases

Error	Status	Description
Duplicate key (11000)	400	Returned when trying to register with an already-used email.
Mongoose ValidationError	400	Mongoose model validation failed; returns joined error

Error	Status	Description
		messages.
JsonWebTokenError	401	Token signature is invalid or cannot be decoded.

Sample Requests & Full Shopping Flow

Step 1 — Login

```
POST /api/auth/login
{ "email": "user@example.com", "password": "pass" }

// Response
{ "msg": "Login successful", "token": "eyJ..." }
```

Step 2 — Browse Products

```
GET /api/products/?category=665cat005&search=oak

// Response
[{ "_id": "...", "name": "Classic Wooden Frame", "price": 29.99 }, ...]
```

Step 3 — Add to Cart

```
POST /api/cart/
{ "productId": "665abc123", "quantity": 2 }

// Response
{ "items": [...], "totalPrice": 59.98 }
```

Step 4 — Checkout

```
POST /api/orders/checkout
{ "shippingAddress": "123 Nile St, Cairo" }

// Response
{ "_id": "orderDocId", "status": "pending", "totalPrice": 59.98 }
```

Admin — Mark Order as Delivered

```
PUT /api/orders/orderDocId/status
Authorization: Bearer <admin_token>
```

```
{ "status": "delivered" }

// Stock of each ordered product is automatically decremented.
```